

Learning reinforcement learning by reproducing CURL

A guide to the experiment, the code, and the evidence we need.

The main idea. We want a small training experiment whose behavior we can explain and check. First we will reproduce an existing method. Then we will remove one component to ask what it contributes.

Where are we now?

We have completed three matched comparisons on one task. **CURL finished higher in all three at the planned 100,000-step endpoint.** Page 8 shows the measured curves. Worked teaching examples are labeled as illustrations; the published reference scores belong to the paper's authors.

After 100,000 simulator steps, the scores were 678 versus 454 in the first pair, 446 versus 241 in the second, and 588 versus 463 in the third, favoring CURL. Three pairs give an encouraging signal, not a firm conclusion or a reproduction of the paper's average. Completed-results cutoff: 4 October, 18:37 UTC.

Longer training is running again after a checkpoint-saving problem in our adapter. The earlier saved state survived. We repaired the format and verified that learning resumed. No 500,000-step result is complete yet. Execution update: 19:42 UTC; pages 6–7 explain the recovery and its extra cost.

Why this paper?

CURL combines two kinds of learning in a compact setting: learning which actions earn reward, and learning useful visual features by matching related images. The authors provide code. This gives us concrete pieces to inspect and compare without starting with a large language-model training system. The paper is *CURL: Contrastive Unsupervised Representations for Reinforcement Learning*, by Laskin, Srinivas and Abbeel, ICML 2020.¹

What this exercise does not cover

This is not an RLM and does not train a language model. It does not use supervised fine-tuning (SFT), where a model imitates supplied answers or actions. Here, self-supervised learning (SSL) means constructing an image-matching task from the agent's own observations. Reinforcement learning (RL) uses rewards from a simulator. A successful reproduction would improve our experimental practice, but would not retroactively prove our earlier RLM training correct.

How to read this guide

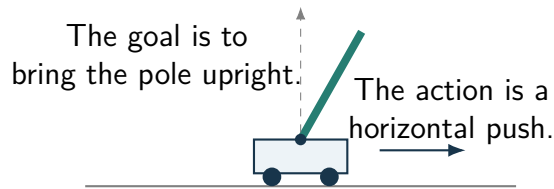
Pages 2–4 explain the task and learning process with examples. Page 5 explains the comparison and scores. Page 6 records implementation findings that affect interpretation. Pages 7–8 give next decisions, checks and the first paired curves. The detailed configuration is in the adjacent research protocol, not hidden in the prose.

¹<https://proceedings.mlr.press/v119/laskin20a.html>

1 / THE TASK

What does the model actually do?

Imagine a cart that moves left and right on a track. A pole is attached to the cart. The controller must move the cart to swing the pole upward and keep it balanced. We use the `cartpole/swingup` task from DeepMind Control Suite, not the similarly named Gym CartPole benchmark.



Conceptual drawing, not an observation from a local simulator run.

The input is pictures, not the correct action.

The learner receives three recent color images together. Several images help it distinguish a pole moving left from one moving right, even when one snapshot looks similar. It is not handed the pole's angle and velocity as the policy's input. A neural network first turns these pixels into a compact numerical description. This part is called the **encoder**.

The **policy**, also called the actor, uses that description to choose a continuous action: how hard to push the cart. Its weights are the numbers changed during training. For this reproduction they start from random initialization, not from a pretrained language model or a demonstration dataset.

The simulator provides feedback.

After a push, the simulator returns new images and a numerical reward. The task's reward function defines which behavior is desirable. The learner does not get a label saying, "the correct action was to push left." It has to learn which actions lead to better outcomes over time.

The main idea. This is trial-and-error learning with feedback, not imitation of an expert's actions. The simulation supplies experiences as training proceeds.

Where is the dataset?

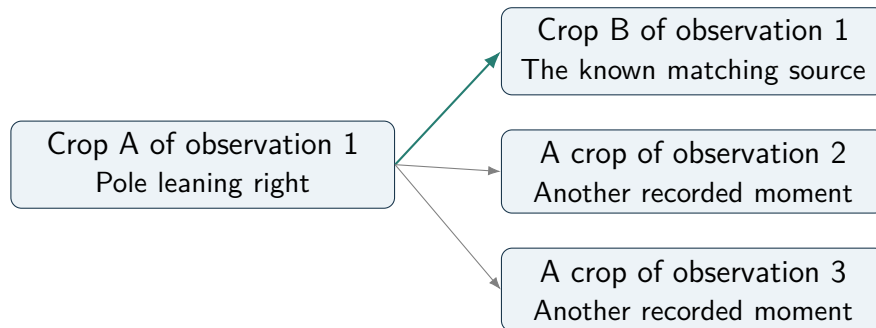
The agent builds it while acting. Each record contains an observation, its action, the resulting reward, the next observation and whether the episode ended. Old records are kept in a **replay buffer**: a memory of experience that can be sampled repeatedly. Evaluation episodes are kept out of that memory.

What is CURL learning from?

The task reward teaches the controller what to accomplish. CURL adds another exercise to help the encoder interpret its pictures. That exercise does not require someone to label the images.

A concrete image-matching example

Take one recorded three-image observation of a pole leaning right. Make two slightly different crops of it. Those two views come from the same observation, so we know that they should match. Views cropped from other recorded observations serve as alternatives. The pictures below are simplified labels, not actual frames.



The matching target comes from how we constructed the pair. It is not an expert action and not the task reward. This is the **self-supervised** part. The method is **contrastive** because it compares a known match with alternatives. Different observations can still look very similar; being a different record does not guarantee a meaningfully different physical situation.

What does one loss calculation mean?

Suppose the model assigns similarity scores 2, 0 and -1 to those three candidates, with the correct match first. Taking the exponential of each score gives positive weights of approximately 7.389, 1 and 0.368. Dividing the matching weight by the total gives

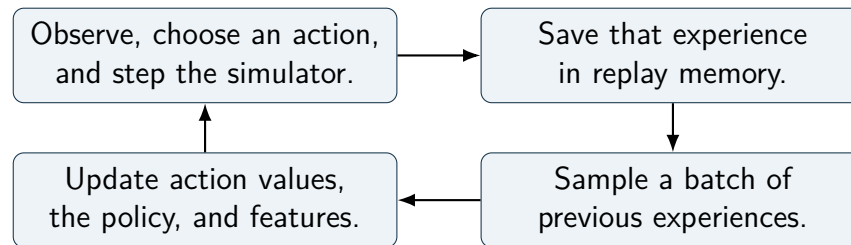
$$\text{Probability of the match} = 7.389 / (7.389 + 1 + 0.368) \\ \text{which is approximately } 0.844.$$

The matching loss is the negative natural logarithm of that probability, approximately 0.170. If all three scores were equal, the probability would be $1/3$ and the loss about 1.099. Lower loss means that this matching exercise is going better. **These are invented teaching numbers, calculated directly, not training measurements.**

The main idea. CURL's hope is that learning to recognize related views helps the controller learn from fewer interactions. A lower image-matching loss alone does not prove better control.

How do the weights get updated?

CURL's control learner is Soft Actor-Critic (SAC). The most useful distinction is between choosing actions and estimating their consequences. The **actor** chooses an action. A **critic** estimates how much future reward that action could lead to. The implementation uses two critics to reduce reliance on an overly optimistic estimate.



What teaches the critic?

For a saved transition, it compares its previous prediction with the reward actually received plus an estimate of what can happen next. In simplified form,

$$\text{Target} = \text{reward now} + \text{discounted estimate of future value.}$$

The discount factor gives less weight to distant rewards. SAC also includes an exploration term and special handling of episode endings; this equation explains the idea, not every term of the implementation.

What teaches the actor and encoder?

The actor is updated toward actions the critics value, while SAC also encourages some action diversity. The encoder receives learning signals from predicting values and from CURL's image-matching exercise. Related target networks change gradually, rather than immediately copying every update. This helps prevent the learner's targets from moving too abruptly. Exact parameter sharing and update order matter; we preserve the authors' implementation before making changes.

What does the computer do during a run?

The CPU advances the simulation and manages experience; the GPU performs neural network calculations. Training begins with random actions to fill replay memory. It then alternates collection with updates. The first protocol uses 1,000 random action decisions and batches of 128 replay records. A single GPU is sufficient for this experiment. Our completed 100,000-step training/evaluation loops took about 13–14.4 minutes each, excluding setup and the final checkpoint write.

The main idea. One interaction and one gradient update are different things. Replay lets the learner revisit old experience. We must record both the amount of experience and the amount of computation.

How will we know whether it worked?

There are two questions, and they need different evidence.

First: can we reproduce a published result?

We start with the authors' CURL implementation on cartpole swing-up. The paper's Table 1 reports the following episode returns. The spread is the standard deviation across ten separately trained seeds, not a confidence interval.

Training experience	Authors' CURL result	Our CURL result
100,000 simulator steps	582 (SD 146)	571 (SD 117; 3 seeds)
500,000 simulator steps	841 (SD 45)	Not complete

We will report any modern software changes and use the published values as reference points, not declare success because one lucky run enters that range. Our first three-seed comparison is smaller than the authors' ten-seed report.

Second: does the extra matching objective help?

Compare CURL with the same learner after removing only the contrastive update. Keep random image crops in both arms. Otherwise, an apparent benefit could come from cropping rather than the extra learning objective. This is our controlled ablation, **not necessarily the paper's "Pixel SAC" baseline**.

What do the axes and score mean?

The plot on page 8 puts **training simulator steps** on the horizontal axis and **mean evaluation episode return** on the vertical axis. An episode return is the sum of its rewards. At each evaluation we average ten episodes without changing the weights. Higher return means better performance under this reward function, not a percentage of questions answered correctly. It shows each training seed, not only a smoothed average.

The main idea. The cartpole setting repeats each chosen action eight times. So 12,500 model decisions normally produce 100,000 simulator steps. Evaluation interactions are counted separately.

We evaluate a separate simulator with fixed seeds and deterministic actions. Ten episodes of one trained policy tell us about that policy; they are not ten independent training runs. We compare the fixed final checkpoint, not the most flattering point on the curve. Time and GPU use are separate measurements: learning from less experience need not mean finishing faster.

What could make a run misleading?

These findings concern the implementation, not which method performs better. The review uses upstream commit 8416d6e3869e38ca0e46fcbc54a2f784dc09d7fc.

A saved model is not necessarily a resumable run.

The training script's `-save_model` path saves CURL's representation module. That does not include the full acting policy and learning state. Another method saves actor and critic weights, but still omits important state.

For resumption, we also need optimizer history, target networks, the learned exploration parameter, replay memory, counters and random-generator states. To continue the same trajectory, we need simulator state and recent frames too. Restarting at a new episode can be useful, but must not be described as an identical continuation. Our CPU test restored the actual upstream agent and reproduced its next fixed-batch update exactly, including shared parameters.

A longer run also exposed a defect in *our* saving code: one replay array grew beyond the old format's size limit. The previous checkpoint survived. We changed the format and tested saving and reloading an array larger than 4 GiB. Small pilot tests had not exercised this limit.

Evaluation should not disturb the training world.

The upstream loop evaluates using the same environment object used for training. If evaluation resets that object halfway through a training episode, the stored observation can stop matching the simulator's actual state. We use a separate evaluation environment and preserved training random state. This is a documented adaptation; it is not evidence that this failure occurred in the paper's runs.

The exact optimizer wiring matters.

Two optimizers in the contrastive update include the same query encoder parameters, and both take a step. A clean-room implementation with only one step would therefore not automatically match this released code. We should first measure that behavior on a fixed batch and preserve it in the reference run. Calling it an established bug, or silently "fixing" it, would be premature.

A quickstart command is not the paper's protocol.

The README's example runs one million model decisions. That is not the paper's 100,000-simulator-step comparison. The old dependencies also need a pinned compatibility stack. We must identify changes to rendering, environment behavior and learning code instead of assuming that a program which starts is a faithful reproduction.

The main idea. The lesson is not to build a huge testing system before learning anything. It is to check the few details that can change the scientific meaning of the result.

These findings come from the authors' MIT-licensed implementation.² The hyperparameters and step terminology are documented in the supplement.³

²<https://github.com/MishaLaskin/curl/tree/8416d6e3869e38ca0e46fcbc54a2f784dc09d7fc>

³<https://proceedings.mlr.press/v119/laskin20a/laskin20a-sup.pdf>

What happens next, and what can we conclude?

The next useful sequence

The short test and all three reference/control pairs have completed at the fixed 100,000-step budget. We are now continuing both versions from their saved states to a saved training history of 500,000 steps. This asks whether the early advantage lasts with more training. Every original model is included, regardless of its score. Resuming a model does not create another independent training run.

The first extension stopped at 409,000 steps because saving failed. Its latest intact state was at 307,000, and learning has resumed there. The lost branch still cost 102,000 interactions. Reaching 500,000 again will therefore cost 602,000 physical training interactions in total. We will report that extra cost and keep the abandoned measurements separate from the resumed curve.

How results will change the plan

If the control catches up, the extra objective may mainly help early learning. If CURL keeps its lead, we will test whether the effect extends to another task. If the ordering reverses, we must revise the story: better early performance does not guarantee better later performance. A falling image-matching loss with flat reward would also remind us that solving the image exercise is not enough.

Our independent small NumPy calculation matches the released Torch loss and gradients to numerical precision. That checks the calculation, not its usefulness for control. We need the matched experiment to answer the latter question.

What has actually been established today?

Item	Status
Paper and core source review	Complete for the first task.
Budget and comparison specified	Yes; modern simulator stack recorded.
GPU training and policy evaluation	Three pairs complete; extensions running.
Checkpoint save/reload verification	Next-update and large-array tests passed.
Evidence that CURL beats the control	Three favorable endpoints on one task.

How will we report the longer comparison?

We will show fixed final scores and curves for each training seed, with failures and recovery costs stated separately. Missing results will remain missing, not become zero scores. There is no completed 500,000-step result yet.

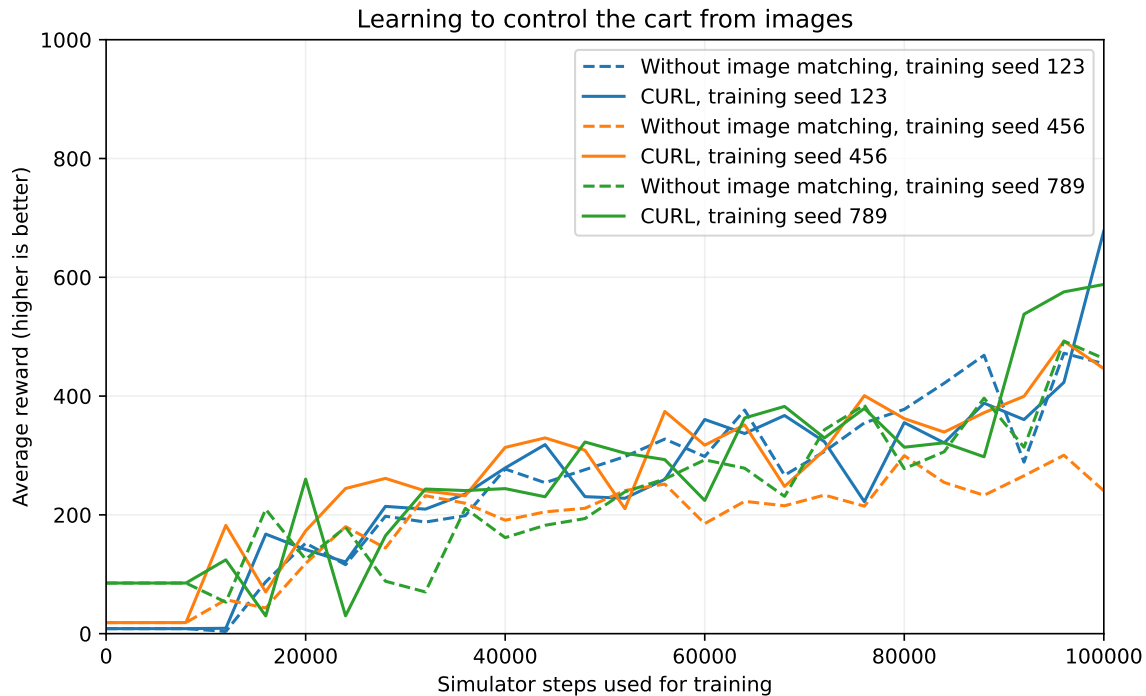
The main idea. Our immediate goal is a trustworthy learning example, not a novelty claim. The transferable skills are controlling comparisons, reading the actual update, evaluating fairly, and preserving enough state to explain and resume a run.

Research records. The configuration and execution handoff are in `experiments/curl_replication_20261004/README.md` and `experiments/curl_replication_20261004/manifest.json`. Earlier RLM work is preserved in `docs/RESEARCH_RESUME_2026-10-04.md`.

7 / THREE PAIRED RESULTS

CURL finished higher in all three comparisons.

At the fixed 100,000-step endpoint, CURL scored **678 versus 454** in the first pair, **446 versus 241** in the second, and **588 versus 463** in the third. Its control uses the same random crops but no image-matching update. Higher reward means better swinging and balancing, not a percentage of successful episodes.



Solid lines are CURL; dashed lines are the control. Each color is a separate training seed, which changes initialization and training randomness. Every point averages ten fixed evaluation starts. There are **three trained models per version**, not thirty independent training runs.

What can we conclude so far?

The average endpoint is **571 for CURL versus 386 for the control**. All three paired differences favor CURL, but the curves sometimes trade places. We report the endpoint chosen before training, not each curve's best point. The control also learned: image matching was helpful here, not necessary for learning. Three seeds on one task do not establish a general advantage.

What did it cost, and what happens next?

Each training/evaluation loop took about 13–14.4 minutes, plus 11–12 seconds to save its checkpoint. Setup was extra. This matches training interactions, not computing time: CURL performs an extra learning update.

Now running: extend all three pairs to 500,000 steps. Does the advantage last, shrink, or reverse with more training? There is no completed 500k result yet.

Evidence. Native records: the six run folders in data/. Task: DeepMind Control Suite cartpole/swingup; training seeds 123, 456, 789; evaluation seeds 10000–10009. Modern simulator compatibility adaptation.